

AskTube AI

Chat with YouTube videos through transcript-grounded AI.
<https://asktube-ai.duckdns.org/>



SEARCH

TRANSCRIPT

RAG

CITATIONS

Netflix-inspired UI + FastAPI + LangChain +
ChromaDB + analytics + EC2

Final Project | Sabeur Zarai | IronHack

YouTube is huge. Learning from long videos is slow.

AskTube AI turns a video into a searchable, conversational knowledge base.

USER FLOW

Search a topic, choose a video, let the app process the transcript, then ask questions with source timestamps.

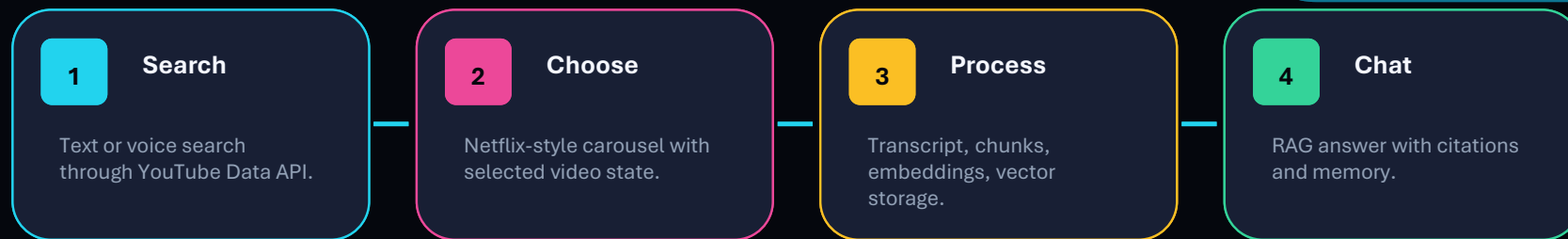
TRUST RULE

The assistant answers from transcript evidence only. If the transcript cannot answer, it refuses instead of inventing.

WHY IT MATTERS

Learners can skip guessing where information lives and jump straight to the exact moment in the video.

From search to cited answer



Cinematic UI: dark mode, glass surfaces, duration filters, smooth transitions, 3D guidance, and graceful loading/error states.

Three layers, one learning loop

FRONTEND

Next.js 14, TypeScript, TailwindCSS, shadcn/ui, Framer Motion, Embla, Three.js, React Three Fiber.

BACKEND API

FastAPI async routes for search, transcripts, chunks, vectorstore, chat, agent, speech, evaluations, and analytics.

AI + DATA

LangChain tools and agent, OpenAI models, ChromaDB vector storage, analytics DB, LangSmith tracing, Whisper fallback.

Docker Compose runs frontend, backend, ChromaDB, and analytics storage together for local development and deployment testing.

A YouTube video becomes retrieval-ready text.

- YouTube Data API retrieves metadata, thumbnails, duration, and channel information, then applies the selected length filter.
- youtube-transcript-api is the primary source for public captions and timestamps.
- Whisper is used only as a fallback when captions are unavailable and the deployment can safely fetch audio.
- LangChain chunking keeps timestamp metadata attached to every chunk.
- OpenAI embeddings are stored in ChromaDB for similarity search.

STATUS STREAM

metadata
transcript
chunking
embeddings
vector_storage
ready / error

The processing screen reflects backend progress, not only decoration.

The chatbot uses tools, not just a single prompt.

AgentService binds StructuredTool objects to the LLM and returns answer, citations, tool steps, and session_id.

search_youtube_videos

extract_transcript

chunk_transcript

store_video_vectors

ingest_video

retrieve_context

answer_question

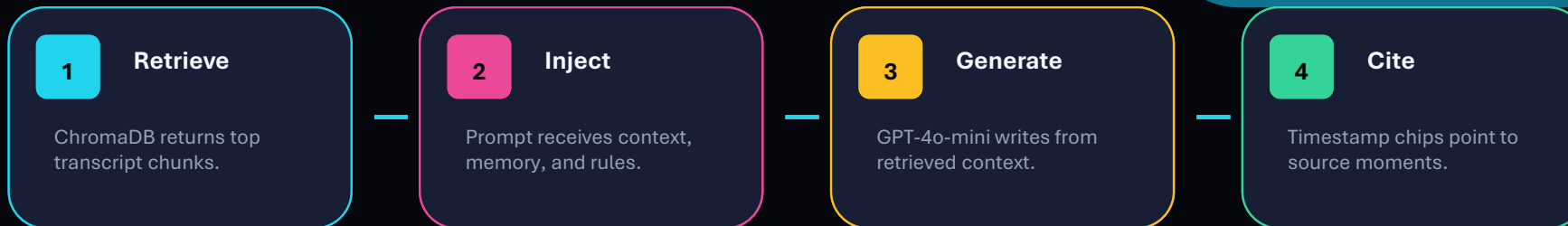
GUARDRAILS

The final answer must use answer_question, stay transcript-only, and include timestamp citations. Off-topic questions should be refused.

MEMORY

Session-based memory keeps follow-up questions coherent.

Answers are grounded in transcript evidence.



Anti-hallucination rule: if the transcript does not contain the answer, AskTube says it cannot answer from the video.

The interface feels like a learning product, not a backend demo.

VOICE SEARCH

Browser Web Speech API captures live dictation with waveform feedback. If it fails, MediaRecorder sends audio to `/api/speech/transcribe`.

ASSISTANT

A floating 3D robot moves through the journey, speaks short guidance, and opens a compact transcript-grounded chat.

ACCESSIBILITY

Semantic labels, visible focus states, reduced-motion handling, screen-reader status messages, and touch-friendly layouts.

Extra polish: loading skeletons, retry states, TTS answer playback, responsive stacking, and collapsible evidence panels.

The platform is measurable, not just usable.

AskTube tracks product usage, RAG quality, pipeline performance, UX behaviour, and operational health.

PRODUCT + UX

search_submitted, video_selected, voice_search_completed, suggested_prompt_clicked, timestamp_clicked, 3d_chatbot_interacted.

AI / RAG

retrieval latency, generation latency, chunks retrieved, token estimates, citation coverage, hallucination warnings.

PIPELINE

transcript time, embedding duration, vector insert/query timing, chunk count, processing duration, Whisper fallback usage.

/analytics

Next.js dashboard with overview, AI, pipeline, UX, business metrics, and recent events.

/metrics

Prometheus metrics for HTTP latency, RAG, embeddings, vector search, processing, and WebSockets.

The project is testable, explainable, and demo-ready.

100+

AUTOMATED TESTS

17

RAG EVAL CASES

0

BEHAVIORAL FAILS

3

DOCKER SERVICES

- Coverage includes routes, services, tools, WebSocket ingest, speech transcription, and RAG behavior.
- Evaluation cases test answerable questions, refusals, citation accuracy, summaries, memory, and hallucination prevention.
- LangSmith tracing can inspect latency, context, answer quality, and chain behavior.

Challenges faced during the project

The hard parts were not only building the chatbot, but making it reliable on real YouTube data and EC2.

1

YouTube access

CHALLENGE

Cloud IPs can be blocked and some videos have no usable captions.

SOLUTION

Use transcript API first, add residential proxy support, and keep Whisper only as fallback.

2

Trustworthy answers

CHALLENGE

A normal LLM can answer from memory and hallucinate outside the video.

SOLUTION

Use RAG guardrails, transcript-only context, refusal behavior, and timestamp citations.

3

Production UX

CHALLENGE

Processing takes time, voice needs HTTPS, and users need clear feedback.

SOLUTION

Add WebSocket progress, Nginx + HTTPS domain, retry states, analytics, and visible errors.

Main lesson: every failure mode became a visible state, fallback, or metric.

Demo path: search, prepare, ask, verify.

- Open the hosted app: <https://asktube-ai.duckdns.org/>
- Search with text or voice and apply a duration filter.
- Prepare one video and show the live ingestion progress.
- Ask one question, then verify the timestamp citations.

WHAT TO EMPHASIZE

Transcript-grounded workflow with tools, memory, vector search, citations, analytics, and polished UX.

Keep the demo focused on trust: every answer links back to the video. EC2 proves deployment with HTTPS; proxy support helps with YouTube cloud-IP limitations.